

Sistema Distribuido de Cidade Inteligente

Detalhes de Implementacao, Mensagens e Tecnologias

Distribuicao de Dados

Universidade Federal do Ceara

2026.1

Objetivo do Sistema

- Implementar um sistema distribuido para coleta, controle e consulta de dados urbanos.
- Integrar tres fontes de dados: clima, qualidade do ar e poste inteligente.
- Centralizar descoberta, leituras, historico, controle e consultas em um Gateway.
- Disponibilizar uma interface web analitica para visualizacao e operacao do sistema.
- Usar mensagens padronizadas com Protocol Buffers entre os processos.

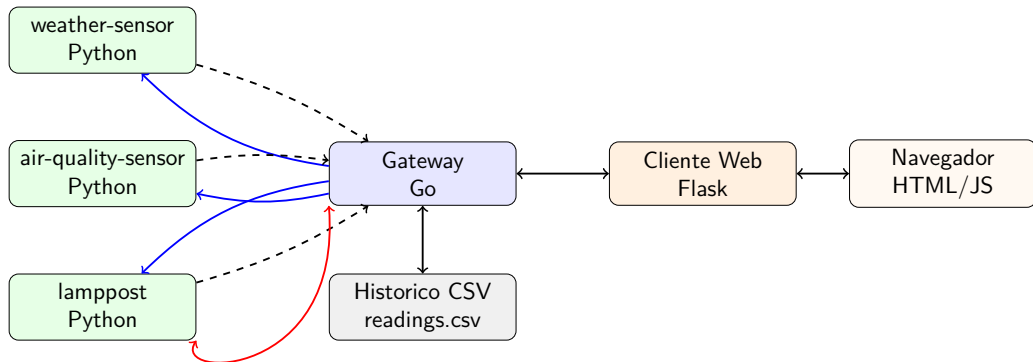
Partes do Trabalho

- 1 Descoberta dinamica das fontes de dados pelo Gateway.
- 2 Recebimento e armazenamento de leituras das fontes.
- 3 Cliente web analitico para visualizacao, controle e consultas agregadas.

Foco da implementacao

Formato das mensagens, comunicacao entre processos, linguagens, bibliotecas e protocolos usados.

Arquitetura Geral



linha azul: discovery UDP multicast 239.0.0.1:9999

linha tracejada: leituras UDP :11000

linha vermelha: controle TCP do lamppost :10002

linha preta: TCP + Protobuf :12000, HTTP :5000 e persistencia CSV

Processo	Linguagem	Papel
Gateway	Go	Descoberta, estado, historico, leituras e TCP
weather-sensor	Python	Sensor de Temperatura e umidade
air-quality-sensor	Python	Sensor de CO2, particulados e indice de ar
lamppost	Python	Fonte controlavel de luminosidade e energia
Cliente Web	Python/Flask	Interface web analitica

- Discovery UDP multicast: 239.0.0.1:9999
- Servidor UDP (Gateway): 11000
- Servidor TCP (Gateway): 12000
- Servidor de Controle TCP (Lamppost): 10002
- Cliente Web Flask: 5000

Parte 1: Descoberta de Fontes

- O Gateway envia uma requisicao UDP multicast para o grupo de descoberta.
- Cada fonte responde por UDP usando `DiscoveryResponse` serializado com Protobuf.
- A resposta informa nome, tipo, endereco de controle e status operacional.
- O Gateway registra as fontes em um estado central protegido por `sync.RWMutex`.
- A descoberta periodica ocorre a cada 5 segundos.
- Fontes conhecidas que nao respondem a uma rodada sao marcadas como `OFFLINE`.

Formato das Mensagens de Descoberta

```
message DiscoveryRequest {  
    string gateway_id = 1;  
    string gateway_ip = 2;  
    int32 readings_port = 3;  
    int32 client_port = 4;  
}  
  
message DiscoveryResponse {  
    string source_name = 1;  
    string source_type = 2;  
    string ip = 3;  
    int32 control_port = 4;  
    bool controllable = 5;  
    string status = 6;  
}
```

Parte 2: Envio de Leituras

- As fontes enviam leituras periodicas ao Gateway por UDP.
- Cada pacote contem o nome da fonte, timestamp e uma leitura tipada.
- O campo oneof permite reaproveitar o mesmo envelope para diferentes sensores.
- O Gateway converte as leituras recebidas em metricas internas com valor, unidade e timestamp.
- As leituras sao mantidas em memoria e persistidas em arquivo CSV.

Envelope de Leituras

```
message ReadingPacket {  
  string source_name = 1;  
  int64 timestamp_unix_ms = 2;  
  
  oneof reading {  
    temperature.TemperatureReading temperature = 3;  
    airquality.AirQualityReading air_quality = 4;  
    lamppost.LamppostReading lamppost = 5;  
  }  
}
```

Vantagem do oneof

O Gateway recebe um unico tipo de pacote, mas preserva o formato especifico de cada fonte.

Formatos de Leituras por Fonte

```
message TemperatureReading {  
    double temperature_celsius = 1;  
    double humidity_percent = 2;  
}  
  
message AirQualityReading {  
    double co2_ppm = 1;  
    double particulate_matter_ug_m3 = 2;  
    double air_quality_index = 3;  
}  
  
message LamppostReading {  
    double luminosity_percent = 1;  
    double energy_consumption_kwh = 2;  
    bool light_on = 3;  
}
```

Métricas Armazenadas pelo Gateway

Fonte	Metrica	Unidade
weather	temperature_celsius	celsius
weather	humidity_percent	percent
air_quality	co2_ppm	ppm
air_quality	particulate_matter_ug_m3	ug/m3
air_quality	air_quality_index	aqi
lamppost	luminosity_percent	percent
lamppost	energy_consumption_kwh	kWh
lamppost	light_on	boolean

Observacao

`light_on` e armazenado como valor numerico booleano: 1.0 para ligado e 0.0 para desligado.

Historico Persistente no Gateway

- O Gateway armazena as leituras em memoria para consultas durante a execucao.
- As leituras tambem sao persistidas em CSV para preservar historico entre reinicializacoes.

Parte 3: Cliente Web Analitico

- Cliente implementado como aplicacao web em Python com Flask.
- O navegador acessa a interface HTTP na porta 5000.
- O backend Flask usa `gateway_api.py` para falar com o Gateway por TCP.
- A comunicacao Flask-Gateway usa `ClientRequest` e `ClientResponse` em Protobuf.
- A tela atualiza os dados periodicamente e apresenta graficos de serie temporal.

Framing TCP

Cada mensagem TCP usa 4 bytes iniciais com o tamanho do payload, seguidos pelos bytes Protobuf.

- Listagem de fontes descobertas e respectivos estados.
- Listagem das leituras recentes recebidas pelo Gateway.
- Filtro por fonte e metrica para visualizacao no grafico.
- Consultas agregadas com AVG, STDDEV, MIN e MAX.
- Painel de controle do lamppost: ligar, desligar, consultar, ajustar luminosidade e simular falha.
- Atualizacao automatica da interface a cada 5 segundos.

Mensagens Cliente-Gateway

A interface web conversa por HTTP com o backend Flask. O backend traduz as operacoes da tela para mensagens TCP/Protobuf enviadas ao Gateway.

```
message ClientRequest {
  oneof request {
    ListSourcesRequest list_sources = 1;
    ListReadingsRequest list_readings = 2;
    SendCommandRequest send_command = 3;
    AggregateRequest aggregate = 4;
  }
}

message ClientResponse {
  bool success = 1;
  string message = 2;

  oneof response {
    ListSourcesResponse list_sources = 3;
    ListReadingsResponse list_readings = 4;
    SendCommandResponse send_command = 5;
    AggregateResponse aggregate = 6;
  }
}
```

Listagem de Fontes e Leituras

```
message SourceInfo {  
    string name = 1;  
    string address = 2;  
    string ip = 3;  
    int32 port = 4;  
    bool controllable = 5;  
    string status = 6;  
    int64 last_seen_unix_ms = 7;  
    string source_type = 8;  
}
```

```
message ReadingInfo {  
    string source_name = 1;  
    string source_type = 2;  
    string metric = 3;  
    double value = 4;  
    string unit = 5;  
    int64 timestamp_unix_ms = 6;  
}
```

- O lamppost é a unica fonte controlavel.
- O Gateway envia comandos por TCP para a porta 10002.
- A fonte responde com status operacional, luminosidade, consumo e estado da luz.
- A fonte tem como atributos configuraveis a luminosidade e o estado da luz.
- O consumo de energia acumulado e exibido a partir das leituras mais recentes.

Mensagens de Controle do Lamppost

```
message LamppostCommand {
  oneof command {
    LamppostTurnOnRequest turn_on = 1;
    LamppostTurnOffRequest turn_off = 2;
    LamppostGetStateRequest get_state = 3;
    LamppostSimulateFailureRequest simulate_failure = 4;
    LamppostSetLuminosityRequest set_luminosity = 5;
  }
}

message LamppostResponse {
  bool success = 1;
  string message = 2;
  bool active = 3;
  string status = 4;
  double luminosity_percent = 5;
  double energy_consumption_kwh = 6;
  bool light_on = 7;
}
```

Retorno de Comandos ao Cliente

```
message SendCommandResponse {  
    bool success = 1;  
    string message = 2;  
    string source_status = 3;  
    double luminosity_percent = 4;  
    bool light_on = 5;  
}
```

- O cliente exibe luminosidade e estado da luz apenas quando o comando tem sucesso.
- Em caso de falha local ou erro TCP, a resposta mostra apenas resultado, mensagem e status.

- As consultas sao feitas por fonte, metrica, operacao e janela temporal.
- Operacoes suportadas: AVG (Média), STDDEV (Desvio Padrão), MIN (Mínimo) e MAX (Máximo).
- O Gateway filtra leituras por `source_name` (Nome da fonte), `metric` (Métrica) e `window_seconds` (Intervalo de tempo em segundos).
- A resposta inclui valor, unidade, quantidade de amostras e janela usada.
- Quando `window_seconds = 0`, todo o historico disponivel e considerado.

Formato das Consultas

```
enum AggregateOperation {
    AGGREGATE_OPERATION_AVG = 0;
    AGGREGATE_OPERATION_STDDEV = 1;
    AGGREGATE_OPERATION_MIN = 2;
    AGGREGATE_OPERATION_MAX = 3;
}

message AggregateRequest {
    string source_name = 1;
    string metric = 2;
    AggregateOperation operation = 3;
    int64 window_seconds = 4;
}

message AggregateResponse {
    string source_name = 1;
    string metric = 2;
    AggregateOperation operation = 3;
    double value = 4;
    int32 sample_count = 5;
    int64 window_seconds = 6;
    string unit = 7;
}
```

- A mensagem `simulate_failure` é enviado ao `lamppost`.
- A fonte responde ao comando e entra em falha temporaria por 15 segundos.
- Durante a falha, o `lamppost` nao responde discovery, nao envia leituras e nao responde mensagens TCP.
- O Gateway marca como OFFLINE quando a fonte nao responde a uma rodada de descoberta.
- Se um comando TCP for enviado durante a falha, o Gateway tambem trata o erro como indisponibilidade da fonte.

Tecnologia	Uso
Go	Gateway, estado central, UDP/TCP, analytics e CSV
Python	Fontes de dados e backend do cliente
Flask	Servidor HTTP do cliente web
HTML/CSS/JavaScript	Interface, atualizacao automatica e grafico
Protocol Buffers	Contratos e serializacao das mensagens
UDP multicast	Descoberta de fontes
UDP unicast	Envio de leituras ao Gateway
TCP	Flask-Gateway e controle do lamppost
CSV	Persistencia do historico de leituras

Organizacao do Codigo

- `contracts/`: arquivos `.proto` compartilhados.
- `gateway/`: descoberta, estado, leituras, cliente TCP, analytics, historico e controle.
- `weather-sensor/`: fonte de temperatura e umidade.
- `air-quality-sensor/`: fonte de qualidade do ar.
- `lamppost/`: Poste de luz, fonte controlavel.
- `client/web_client.py`: servidor Flask do cliente web.
- `client/gateway_api.py`: camada TCP/Protobuf entre Flask e Gateway.
- `client/templates/` e `client/static/`: interface web.

- O sistema integra processos independentes com comunicacao UDP e TCP.
- Protocol Buffers padroniza as mensagens entre Go e Python.
- O Gateway centraliza estado, leituras, historico, controle e consultas agregadas.
- O cliente web permite visualizar series temporais, consultar agregados e controlar a fonte atuadora.
- A fonte controlavel permite demonstrar comandos, estado operacional e falhas temporarias.